Chapter05

EDU-100 W5 검수 자동화
시나리오 자동 테스트부터 산출물 현행화까지

AP-Framework V0.42 · 시나리오 자동 테스트 · n8n+Slack 결과 공유 · 산출물 현행화

로디 박한근

Chapter05.
W5 검수 자동화 —
시나리오 자동 테스트부터 산출물 현행화까지

1. 이번 주 한눈에 보기

오늘의 흐름 — 한 줄 파이프라인

1. 강의 — 테스트 전략 · 자동 테스트 · 산출물 현행화
2. 실습1 — 무엇을 검증할지 시나리오 정의 (P-501)
3. 실습2 ★ — /auto-scen-test로 배포 UI 자동 판정 (P-507)
4. 실습3 — n8n + Slack으로 검수 결과 자동 공유
5. 실습4 — 사람·AI의 라이브 육안 검증 (P-505)
6. 실습5 — 바뀐 코드를 설계에 역반영 (현행화, P-508)
7. 실습6 — 결과 종합 → 완료보고서 · Chaining 17/17 (P-504+P-208)
8. 실습7 — 발표 · Phase 4 Gate 인증 · 수료

테스트 코드 0줄로 검수→공유→현행화→완료보고서→수료까지 — 5주 여정을 닫는다

3막으로 보면

막	묶음	무엇을 하나 (산출물)
1막	검증하고 공유	시나리오 정의(P-501) → /auto-scen-test 자동 판정(P-507) → n8n+Slack 결과 자동 공유 · 결과 XLSX·스크린샷 증적
2막	라이브 검증·현행화	사람·AI 라이브 육안 검증(P-505) → 코드→설계 역반영(현행화, P-508)
3막	마무리·수료	완료보고서(P-504+P-208, Chaining 17/17 완성) → 발표·Phase 4 Gate 인증·수료 (.progress 100%)

기억할 핵심 2가지

1. 검증은 2계층이다

① /auto-scen-test(시나리오 기반 자동 테스트, 증적 자동 수집) + ② P-505(라이브 육안 검증). 코드 한 줄 없이 두 계층이 서로 다른 결함을 잡는다.

2. 5주차 = 마지막 주

이번 주를 끝내면 Document Chaining 17/17 완성 + Phase 4 Gate 통과로 5주 여정이 닫힌다 (.progress.md 100% = 수료).

2. 수업 개요

수업 개요

항목	내용
주차 / 총시간	Week 5 / 5 · 300분 (강의:실습 ≈ 20:80)
사전 조건	App Router 풀스택 단일 구성 + Supabase Auth + 라이브 배포 1회 (Phase 3 Gate)
테스트 방식	시나리오 기반 자동 테스트(/auto-scen-test) — 단위 테스트 코드 미작성
프레임워크	AP-Framework V0.42 (App Router + Supabase Auth + 산출물 현행화)
Phase 4 Gate	코드 동작 + 라이브 URL + 자동 테스트 Pass 80%+ + 산출물 현행화 + 완료보고서

학습 목표 (핵심 9가지)

1. 기능명세·API스펙 기반 테스트 시나리오를 체계적으로 생성·정밀화 (P-501)
2. 시나리오를 /xlsx로 자동 테스트용 엑셀로 변환
3. /auto-scen-test로 배포 UI 자동 구동 — 코드 없이 Pass/Fail/Block + 증적
4. n8n 검수 워크플로(가입·임포트) + Slack Webhook으로 검수 결과 Slack 자동 공유
5. P-505 배포 후 검증으로 자동 테스트가 못 잡는 환경 이슈 점검
6. P-504 + P-208로 Document Chaining 17/17 완성
7. 구현 중 바뀐 사양을 설계에 역반영 — xlsx·sb-creator 현행화 (P-508)
8. (보충/과제) P-506 온디맨드 리포트로 파생 자료 자동 생성
9. Phase 4 Gate 6항목 충족 → .progress.md 100% 수료 + 결과 발표

핵심 도구

도구	이번 주차 용도
/auto-scen-test	시나리오 엑셀 기반 웹 UI Playwright 자동 구동 → Pass/Fail/Block + 결과 XLSX
/xlsx	시나리오 → 엑셀 변환 / 기능명세·정보구조·API명세 엑셀 현행화
/sb-creator	화면설계서 HTML 생성 → 편집 가능 PPTX 현행화
n8n + Slack	검수 워크플로 임포트 + Slack Incoming Webhook → 검수 결과 케이스별 Slack 자동 공유
Vercel	App Router 풀스택 단일 프로젝트 배포 (라이브 검수 대상)
NotebookLM	전 생명주기 산출물 소스 등록 · 크로스 문서 질의

프롬프트 라이브러리 (Week 5)

P-ID / 자산	이름	실습
P-501	테스트 시나리오 생성기	실습 1
P-507	시나리오 자동 테스트 (auto-scen-test)	실습 2 ★
n8n 워크플로	시나리오 검수 + Slack 자동 공유	실습 3
P-505	배포 후 검증 자동화	실습 4
P-508	산출물 현행화 (xlsx + sb-creator)	실습 5
P-504 / P-208	테스트결과보고서 / 완료보고서	실습 6
P-506	온디맨드 리포트 생성기	보충/과제

P-507·P-508은 V0.42 신규. 단위 테스트 코드 생성기(이전 P-502)는 제거 — 테스트는 /auto-scen-test로 수행

핵심 개념 — 검수 2계층

① 시나리오 자동 테스트

/auto-scen-test

- 배포 UI를 시나리오대로 자동 구동
- 입력→응답 흐름, 권한 검증
- Playwright · 증적 스크린샷+XLSX
- 코드 0줄로 기능 흐름 검증

② 라이브 육안 검증

P-505

- 레이아웃·에셋·환경 의존 이슈
- 시각적 깨짐, 외부 결제 위젯
- 체크리스트 + 사람·AI 점검
- 자동 테스트가 못 잡는 결함

두 계층이 서로 다른 결함을 잡는다 · 체이닝: 기능명세 → 테스트시나리오(#15) → 자동테스트결과 → 테스트결과보고서(#16) → 완료보고서(#17)

3. 강의 — 테스트 전략 · 자동 테스트 · 현행화

왜 테스트 코드를 안 짜는가

AI가 생성·수행 (80%)

- 정상/비정상/경계값 시나리오
- 시나리오 엑셀 변환(/xlsx)
- 자동 구동 + Pass/Fail/Block 판정
- n8n 검수 워크플로 실행 + Slack 결과 전송
- 현행화 초안(코드→설계 역반영)

PM이 판단 (20%)

- 테스트 전략·우선순위
- UI로 검증할 케이스 선택
- 자동 판정 오탐 검토, 최종 합격
- 어느 채널에 공유·대응 판단
- 설계 의도 vs 구현 정합성

PM은 '무엇을 검증할지' 시나리오로 정의하고, 실행은 /auto-scen-test가 실제 화면을 자동 테스트한다 — 테스트 코드는 작성하지 않는다

시나리오 자동 테스트 동작 원리

1. 시나리오 엑셀 (id / scenario / query / checkpoint)
2. 입력창에 query 입력 → 전송 → 응답·화면 안정화 대기
3. 응답 텍스트 추출 + 전체 페이지 스크린샷
4. checkpoint 키워드로 Pass / Fail / Block 자동 판정
5. 결과 XLSX 기록(결과·일시·응답요약·증적) + 결과 요약 시트

4. 실습 — AP-Rodi-Shop (App Router)

실습 1 · P-501 테스트 시나리오 정밀화 (30분)

목표 기능명세 + API스펙 → 테스트 시나리오 60건 생성·정밀화. UI 입력값과 검증 포인트(checkpoint)가 명확해야 실습 2에 바로 사용 (Chaining Step 15/17)

구분	정상	예외	소계
회원 인증 (Supabase Auth)	5	7	12
상품·장바구니·결제·라이브러리	14	16	30
관리자 (상품·주문·대시보드)	7	5	12
인증/권한 가드 + 통합	4	2	6
합계	**30**	**30**	**60**

포인트: 정상/비정상/경계값 · 인증·권한 가드 · 통합 시나리오 · checkpoint는 화면 실제 문구로

실습 2 ★ · /auto-scen-test 시나리오 자동 테스트 (55분)

1. 테스트시나리오 → /xlsx로 자동 테스트용 엑셀 변환 (id·query·checkpoint)
2. 환경 설치 + 헤더 매핑(--map-check) + 셀렉터 탐색(--discover)
3. 소수 케이스 스모크 테스트 (--limit 3) — 셀렉터·대기·판정 룰 확인
4. 전체 자동 테스트 실행 → Pass/Fail/Block + 결과 XLSX + 스크린샷
5. Fail 분석 — 오탐(checkpoint 수정) vs 실제 버그(수정·재배포·재실행)

테스트 코드 0줄 · Pass Rate 80% 이상이면 합격 · 결과 XLSX는 실습6 P-504에 그대로 인용

auto-scen-test 전체 실행 (CLI)

터미널 Antigravity 통합 터미널

```
cd ~/.claude/skills/auto-scen-test/scripts

# (최초 1회) 의존성 설치
python3 -m pip install --user playwright openpyxl
python3 -m playwright install chromium

# 전체 자동 테스트 실행 (배포 URL 대상)
python3 run_tests.py \
  --scenario-file "<자동테스트_시나리오.xlsx>" \
  --result-file   "<자동테스트결과.xlsx>" \
  --shots-dir     "<자동테스트_스크린샷>" \
  --url           "https://ap-rodi-shop.vercel.app"

# 결과: Pass: n | Fail: n | Block: n | Total: n
#  + 결과 XLSX(결과·일시·응답요약·증적 임베드) + 결과 요약 시트
```

실습 2 결과 예시 — AP-Rodi-Shop

항목	수치
총 케이스	42건 (UI 검증 가능 시나리오)
Pass	36건 (85.7%)
Fail	4건 (검증 키워드 불일치 / 실제 버그)
Block	2건 (외부 결제 위젯 진입 불가)
증적	케이스당 스크린샷 1장 (XLSX 셀 임베드)

합격 기준 Pass Rate 80% 충족 → 합격. 시각적 깨짐·환경 이슈는 P-505 라이브 검증으로 보완

실습 3 · n8n + Slack 테스트 결과 자동 공유 (25분)

1. n8n Cloud 가입 → 무료 인스턴스 (공개 URL만 검수 가능, localhost 불가)
2. Slack Incoming Webhook 앱 설정 → 채널 선택 → Webhook URL 복사
3. 워크플로 JSON 임포트 → Slack 노드 URL을 본인 Webhook으로 교체 → Active
4. 폼 실행 — 대상 사이트 URL + 테스트시나리오.md 첨부 → 제출
5. Slack에 케이스별 PASS/FAIL + 요약 1건 자동 전송 확인

코드 0줄 · 시나리오 .md의 N8N-TEST-MATRIX 표를 파싱해 API 검수 → 결과를 팀 채널에 자동 공유

실습 4 · P-505 배포 후 검증 체크리스트 (20분)

#	검증 항목	발견 가능 이슈
1	회원가입 (특수문자 비밀번호)	Vercel Body Escaping (\! invalid escape)
2	로그인 + Supabase 세션 쿠키	@supabase/ssr 쿠키 미설정, 리다이렉트 루프
3	상품 이미지 표시 (HTTP 200)	정적 에셋 / Storage 버킷 누락
4	UI 중복·깨짐 (레이아웃)	검색창 중복, 모바일 깨짐
5	API 응답 (Network 탭)	RLS 정책, 환경변수, 라우팅 오류
6	환경변수 (대시보드)	SUPABASE URL/KEY, SERVICE_ROLE, SITE_URL 누락

실습 5 · P-508 산출물 현행화 — 역방향 설계 (30분)

/xlsx — 명세 현행화

· 기능명세서.xlsx
· 정보구조도.xlsx
· API명세.xlsx
→ '변경/추가/삭제' 현행화 이력

/sb-creator — 화면 현행화

· 현행 화면 → HTML 생성
· 우상단 버튼 → 편집 가능 PPTX
· PPTX-safe 작성 + 3중 CDN 폴백

완료보고서(실습6) 전에 현행화 먼저 — "코드 = 진실, 문서가 코드를 따라간다" (PM의 설계-코드 정합성 책무)

실습 6 · P-504 + P-208 완료보고서 (Chaining 17/17) (25분)

1. P-504 테스트결과보고서 (#16) — 자동테스트결과.xlsx + 배포 검증 종합
2. P-208 완료보고서 (#17) — 착수보고서 계획 대비 최종 실적·교훈
3. 합격 판정: Pass Rate 80% 충족 + 배포 검증 통과 → Pass
4. GitHub 커밋 + .progress.md 100% 갱신 → Phase 4 Gate Pass 인증

Document Chaining 17/17 = 관리5 + 기획7 + 구현4 + 검수2 + 완료1 = 19건 산출

실습 7 · 발표 및 수료 (40분)

1. 프로젝트 결과 발표 (3~4명 × 5분) — 라이브 URL + 검수 결과(Slack) + 어려웠던 점
2. Phase 4 Gate-Check 6항목 최종 확인
3. .progress.md 100% 갱신 → Phase 4 Gate Pass = 수료
4. 5주 회고 + EDU-200 안내 + 수료증 / 수료 설문

(보충/과제) P-506 온디맨드 리포트(배포가이드 등 파생 자료)는 시간·관심 있는 수강생이 과제로 수행

보충 · 배포 후 검수 실전 버그 3사례

1. 회원가입 500 — Vercel Body Escaping (! → \! invalid escape). 특수문자 케이스로 잡힘
2. 로그인 후 즉시 로그아웃 — @supabase/ssr 쿠키 / middleware 세션 갱신 누락
3. 이미지 404 · 검색창 2개 — Storage/에셋 누락, AI 코드 중복. 시각적 깨짐은 사람이 봐야

"자동 테스트 Pass = 출시 준비 완료"가 아니다 — 자동 테스트(키워드/흐름) + 라이브 검증(시각/환경)은 상호 보완

심화 · 콘텐츠 자동 생성 & 배포 파이프라인

검수 자동화가 '바뀐 것을 검증'했다면, 콘텐츠 자동화는 '없던 것을 매일 만들어 배포'한다 (별개 트랙).

```
Cron(매일 09:00) → Supabase 도서 조회 → 다음 도서 결정 → Gemini 콘텐츠 생성
→ 파싱·base64 → GitHub 06.콘텐츠 저장 → Supabase 업서트 → Slack 알림
```

1. 콘텐츠 전략(무엇을·어떤 순서로)은 사람이 설계 (BOOK_PLAN)

2. 자동화 깨짐은 대부분 'LLM 응답 파싱' — 방어 코드 필수

3. 무인 반복 실행은 멱등(GitHub sha · DB 업서트) · 관측성(Slack 알림)

5. 마무리 · 수료

Document Chaining 17/17 완성

단계 묶음	건수	Phase Gate
관리 (착수·WBS·주간·중간·완료)	5건	Phase 1 통과
기획 (마켓~화면설계서)	7건	Phase 2 통과
구현 (인프라·시스템·DB·디자인)	4건	Phase 3 통과
검수 (테스트시나리오·결과보고서)	2건	Phase 4 통과
완료보고서	1건	.progress.md 100%

총 19건 산출 · 검수 2계층 + 산출물 현행화 + n8n→Slack 결과 자동 공유 + NotebookLM 27건+

5주차 최종 과제 · 평가 기준 (수료 조건)

수준	기준	배점
기본 (C)	테스트시나리오(TC 20+) + 완료보고서 초안	60점
보통 (B)	+ /auto-scen-test 1회(증적) + n8n+Slack 결과 자동 공유 + P-505 배포 검증	80점
우수 (A)	+ Pass Rate 80%+ · 보고서 3종 · 산출물 현행화(xlsx+PPTX) · (선택) P-506	100점

제출: GitHub URL + NotebookLM URL + 배포 URL · 최종 포함 (검수문서 · n8n+Slack 캡처 · 현행화 · 발표 자료 등)

강사 핵심 강조 6가지

1. 테스트 코드 0줄 — PM은 시나리오, 실행은 /auto-scen-test
2. 검수 2계층 — 자동 테스트 + 라이브 육안 검증
3. W4 정합 — App Router + Supabase Auth (Express/Jest/JWT 없음)
4. 산출물 현행화 — 코드를 진실로, 문서가 코드를 따라간다 (완료보고서 전에)
5. 검수 결과 자동 공유 — n8n + Slack으로 팀이 실패를 즉시 인지
6. Phase 4 Gate-Check 6항목 통과 → .progress.md 100% 수료

W5 핵심 메시지 5가지

1. 테스트 코드 없이 시나리오로 기능 검증을 자동화한다
2. 검수는 2계층 — 자동 테스트 + 라이브 검증이 서로 다른 결함을 잡는다
3. 구현하면 사양이 바뀐다 — 현행화로 설계-코드 정합을 유지 (완료보고서 전에)
4. 검수 결과는 n8n+Slack으로 자동 공유 — 팀이 실패를 즉시 인지한다
5. 17/17 Document Chaining = 재현 가능한 AI 기반 PM 워크플로

Thank you

phk4568@naver.com

EDU-200에서 더 깊은 자동화로 만나요